12
Grids

> It is not that polar coordinates are complicated, it is just that Cartesian coordinates are simpler than they have a right to be. – Kleppner and Kolenhow, "An Introduction to Mechanics"

Grids underlie a wide variety of natural structures. Chessboards are grids. City blocks are typically arranged on a grid; indeed, the most natural grid distance measure grid is often called the "Manhattan" distance. The system of longitude and latitude defines a grid over the earth, albeit on the surface of a sphere instead of the plane.

Grids are ubiquitous because they are the most natural way to carve space up into regions so that locations can be identified. In the limit, these cells can be individual points, but here we deal with coarser grids whose cells are big enough to have a shape. In *regular* grids, each of these shapes is identical, and they occur in a regular pattern. Rectangular or *rectilinear* subdivisions are the most common grids, due to their simplicity, but triangle-based *hexagonal* grids are also important. Indeed, the honey industry has exploited the efficiency of hexagonal grids for literally millions of years.

12.1 Rectilinear Grids

Rectilinear grids are familiar to anyone who has used a piece of graph paper. In such grids, the cells are typically defined by regularly spaced horizontal and vertical lines. Non-uniform spacing still yields a regular topology, although the size of each cell may differ. Three-dimensional grids are formed by connected regularly spaced layers of planar grids with perpendicular lines across the layers. Three-dimensional grids also have planar faces, defined between any two face-neighboring cubes.

There are three important components of the planar grid: the vertices, the edges, and the cell interiors. Sometimes we are interested in the interiors of the cells, as in geometric applications where each cell describes a region in space. Sometimes we are interested in the vertices of the grid, such as in addressing the pieces on a chessboard. Sometimes we are interested in the edges of the grid, such as when finding routes to travel in a city where buildings occupy the interior of the cells.

Vertices in planar grids each touch four edges and the interiors of four cells, except for vertices on the boundaries. Vertices in 3D grids touch on six edges and eight cells. In d-dimensions, each vertex touches $2d$ edges and 2^d cells. Cells in a planar grid each touch eight faces, four diagonally through vertices and four through edges. Cells in a 3D grid each touch 26 other cells, sharing a face with 6 of them, an edge with 12 of them, and just a vertex with the other 8.

12.1.1 Traversal

It is often necessary to traverse all the cells of an $n \times m$ rectilinear grid. Any such traversal can be thought of as a mapping from each of the nm ordered pairs to a unique integer from 1 to nm. In certain applications the order matters, such as in dynamic programming evaluation strategies. The most important traversal methods are —

- *Row Major* — Here we slice the matrix between rows, so the first m elements visited belong to the first row, the second m elements to the second row, and so forth. Such an ordering is used inside most modern programming language compilers to represent two-dimensional matrices as a single linear array.

```
(1,1)      row_major(int n, int m)
(1,2)      {
(1,3)              int i,j;        /* counters */
(2,1)
(2,2)              for (i=1; i<=n; i++)
(2,3)                      for (j=1; j<=m; j++)
(3,1)                              process(i,j);
(3,2)      }
(3,3)
```

- *Column Major* — Here we slice the matrix between columns, so the first n elements belong to the first column, the second n elements to the second column, and so forth. This can be done by interchanging the order of the nested loops from row-major ordering. Knowing whether your compiler uses row-major or column-major ordering for matrices is important when optimizing for cache performance and when attempting certain pointer-arithmetic operations.

```
(1,1)       column_major(int n, int m)
(2,1)       {
(3,1)               int i,j;        /* counters */
(1,2)
(2,2)               for (j=1; j<=m; j++)
(3,2)                       for (i=1; i<=n; i++)
(1,3)                               process(i,j);
(2,3)       }
(3,3)
```

- *Snake Order* — Instead of starting each row from the first element, we alternate the order of the directions we travel down the rows. The effect is that of a typewriter which can type both left-to-right and right-to-left so as to minimize printing time.

```
(1,1)       snake_order(int n, int m)
(1,2)       {
(1,3)               int i,j;        /* counters */
(2,3)
(2,2)               for (i=1; i<=n; i++)
(2,1)                   for (j=1; j<=m; j++)
(3,1)                        process(i, j + (m+1-2*j) * ((i+1) % 2));
(3,2)       }
(3,3)
```

- *Diagonal Order* — Here we march up and down diagonals. Note that an $n \times m$ grid has $m + n - 1$ diagonals, each with a variable number of elements. This is a trickier task than it appears at first glance.

```
(1,1)       diagonal_order(int n, int m)
(2,1)       {
(1,2)               int d,j;        /* diagonal and point counters */
(3,1)               int pcount;     /* points on diagonal */
(2,2)               int height;     /* row of lowest point */
(1,3)
(4,1)               for (d=1; d<=(m+n-1); d++) {
(3,2)                       height = 1 + max(0,d-m);
(2,3)                       pcount = min(d, (n-height+1));
(4,2)                       for (j=0; j<pcount; j++)
(3,3)                               process(min(m,d)-j, height+j);
(4,3)               }
            }
```

12.1.2 Dual Graphs and Representations

Two-dimensional arrays are the natural choice to represent planar rectilinear grids. We can let `m[i][j]` denote either the (i, j)th vertex or the (i, j)th face, depending on which

we are interested in. The four neighbors of any cell follow by adding ± 1 to either of the coordinates.

A useful concept in thinking about problems on planar subdivisions is that of the *dual graph*, which has one vertex for each region in the subdivision, and edges between the vertices of any two regions which are neighbors of each other.

The four-color theorem states that every planar map can be colored using at most four colors such that all pairs of neighboring regions are colored differently. However, it *really* is a statement about the number of vertex colors needed to color the dual graph of the map. In fact, the dual graph of any planar subdivision must in itself be a planar graph. Can you show why?

Observe that the dual graphs of both rectangular and hexagonal lattices are slightly smaller rectangular and hexagonal lattices. This is why whatever structure we use to represent vertex connectivities can also be used to represent face connectivities.

An adjacency representation is the natural way to represent an edge-weighted rectilinear grid. This might be most easily done by creating a three-dimensional array `m[i][j][d]`, where d ranges over four values (north, east, south, and west) which denote the edge directions from point (i, j).

12.2 Triangular and Hexagonal Grids

The other regular grids of importance are the *triangular* and *hexagonal* lattices. These are very closely related – indeed, hexagonal grids are essentially triangular grids with alternate vertices removed.

12.2.1 Triangular Lattices

Triangular lattices are constructed from three sets of equally spaced lines, consisting of a horizontal "row" axis, a "column" axis 60° from horizontal, and a "diagonal" axis 120° from horizontal. See Figure 12.1 for an example. Vertices of this lattice are formed by the intersection of three axis lines, so each face of the lattice is an equilateral triangle. Each vertex v is connected to six others, those immediately above and below v on each of the three axes.

To manipulate triangular lattices, we need to be able to identify the proper neighbors of each of these vertices as well as their geographical positions. Doing this requires keeping track of two types of coordinate systems:

- *Triangular/Hexagonal Coordinates* — Here, one vertex is designated as the origin of the grid, point $(0, 0)$. We must assign the coordinates such that the logical neighbors of each vertex are easily obtainable. In a standard rectilinear coordinate system, the four neighbors of (x, y) follow by adding ± 1 to either the row or column coordinates.

  Although the intersection of three lines defines each grid vertex, in fact two dimensions suffice to specify location. We will use the row and column dimensions to define our coordinate system. See Figure 12.1 for an example. A vertex (x, y)

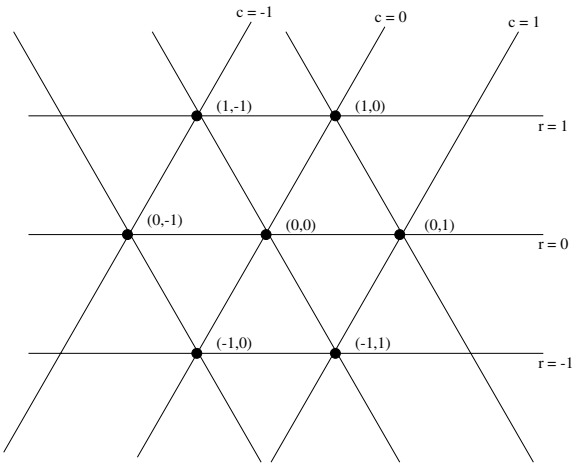


Figure 12.1. A row-column coordinate-system for triangular grids.

lies x rows above the origin, and y (60°)-columns to the right of the origin. The neighbors of a vertex v can be found by adding the following pairs to the coordinates of v, in counterclockwise order: $(0, 1)$, $(1, 0)$, $(1, -1)$, $(0, -1)$, $(-1, 0)$, and $(-1, 1)$.

Similar coordinates can be used to represent the faces of the triangular grid, since the dual graph of the faces will also be a triangular grid. Other coordinate choices are also possible; see [LR76] for an alternative.

- *Geometrical Coordinates* — The vertices of a regular triangular grid also correspond to geometric points in the plane. Note that the grid vertices occur in half-staggered rows in Figure 12.1, because the axis lines are 60° instead of 90° as in rectilinear grids.

  Assume that each lattice point is a distance d from its six nearest neighbors, and that point $(0, 0)$ in triangular coordinates in fact lies at geometric point $(0, 0)$. Then triangular-coordinate point (x_t, y_t) must lie at geometric point

$$(x_g, y_g) = (d(x_t + (y_t \cos(60^o))), dy_t \sin(60^o))$$

  by simple trigonometry. Since $\cos(60^o) = 1/2$ and $\sin(60^o) = \sqrt{3}/2$, we don't even need to use trigonometric functions for this computation.

We will give code for similar manipulations with hexagonal lattices in the next section.

12.2.2 Hexagonal Lattices

Deleting every other vertex from a triangular lattice leaves us with a *hexagonal* lattice, shown in Figure 12.2. Now the faces of the lattice are regular hexagons, and each

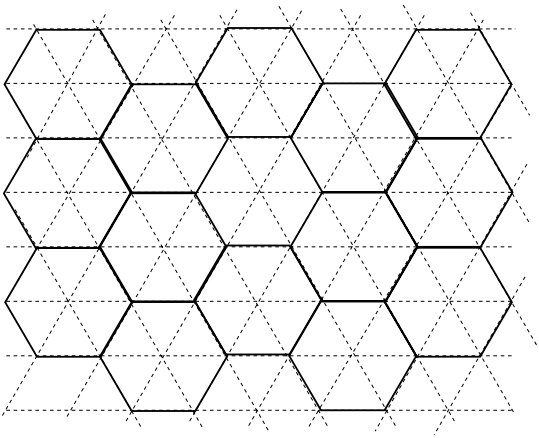


Figure 12.2. Deleting alternate vertices from a triangular lattice yields a hexagonal lattice.

hexagon is adjacent to six other hexagons. The vertices of the lattice now have degree 3, because this lattice is the dual graph of the triangular lattice.

Hexagonal lattices have many interesting and useful properties, primarily because hexagons are "rounder" than squares. Circles enclose the largest amount of area per unit perimeter, so they are the most efficient structures to build. Hexagonal lattices are also more rigid than rectilinear grids, another reason why bees make their honeycombs out of hexagons. Minimizing boundary-per-unit area is also important in minimizing visual artifacts in graphics, so many computer games make images from hexagonal tiles rather than square tiles.

In the previous section, we discussed conversion between triangular/hexagonal coordinates and geometrical coordinates. We assume that the origin of both systems is the center of a disk at $(0, 0)$. The hexagonal coordinate `(xh,yh)` refers to the center of the disk on the horizontal row `xh` and diagonal column `yh`. The geometric coordinate of such a point is a function of the radius of the disk r, half that of the diameter d described in the previous section:

```
hex_to_geo(int xh, int yh, double r, double *xg, double *yg)
{
        *yg = (2.0 * r) * xh * (sqrt(3)/2.0);
        *xg = (2.0 * r) * xh * (1.0/2.0) + (2.0 * r) * yh;
}

geo_to_hex(double xg, double yg, double r, double *xh, double *yh)
{
        *xh = (2.0/sqrt(3)) * yg / (2.0 * r);
        *yh = (xg - (2.0 * r) * (*xh) * (1.0/2.0) ) / (2.0 * r);
}
```

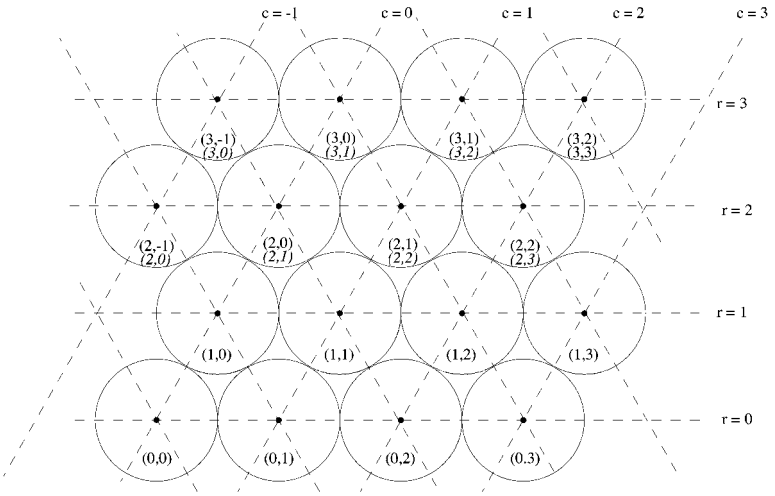


Figure 12.3. A disk packing with hexagonal coordinates, as well as differing array coordinates (below in italics).

The row-column nature of the hexagonal coordinate system implies a very useful property, namely that we can efficiently store a patch of hexagons in a matrix `m[row][column]`. By using the index offsets described for triangular grids, we can easily find the six neighbors of each hexagon.

There is a problem, however. Under the hexagonal coordinate system, the set of hexagons defined by coordinates (hx, hy), where $0 \leq hx \leq x_{\max}$ and $0 \leq hy \leq y_{\max}$, forms a diamond-shaped patch, not a conventional axis-oriented rectangle. However, for many applications we are interested in rectangles instead of diamonds. To solve this problem, we define array coordinates so that `(ax,ay)` refers to the position in an axis-oriented rectangle with $(0, 0)$ as the lower-left-hand point in the matrix:

```
array_to_hex(int xa, int ya, int *xh, int *yh)
{
        *xh = xa;
        *yh = ya - xa + ceil(xa/2.0);
}

hex_to_array(int xh, int yh, int *xa, int *ya)
{
        *xa = xh;
        *ya = yh + xh - ceil(xh/2.0);
}
```

Figure 12.3 shows a hexagonal lattice where each vertex is labeled with both its hexagonal coordinates and, below in italics, its array coordinates if they differ.

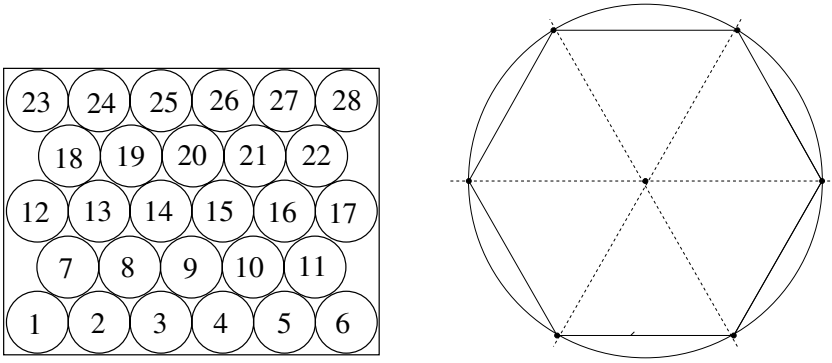


Figure 12.4. Packing plates in a box (l). Viewing a hexagonal grid as a disk packing (r).

12.3 Program Design Example: Plate Weight

A manufacturer of dinner plates seeks to enter the competitive campus dining hall market. Dining halls only buy plates in a single standard size, so they all stack neatly together. Students tend to break a lot of plates during food fights, so selling replacements can be a lucrative business. It is a very cost-sensitive market, however, since the administration gets quite tired of buying extra plates.

Our company seeks an edge in the market through its unique packaging technology. They exploit the fact that hexagonal lattices are denser than rectangular lattices by packaging the plates in $l \times w$ boxes as shown in Figure 12.4(l). Each plate is r units in radius, and the lowest row contains exactly $p = \lfloor w/(2r) \rfloor$ plates. Rows either always contain p plates or alternate between p and $p-1$ plates, depending on the relationship between w and r. Each plate is assigned a unique identification number as shown in Figure 12.4(l). As many layers as possible are put in the box, subject to its length limit l.

Management needs to know how many plates fit in a box of the given dimensions. They also need to know the maximum number of plates resting on top of any given plate to ensure that the packaging is strong enough to prevent breakage (if the post office will break the plates, who needs the students?). Your job is to identify which plate has the most stress and how many plates lie on top of it, as well as the total number of plates and layers in a box of given dimension.

Solution starts below

The first problem is figuring how many rows of plates fit in the box under the hexagonal grid layout. This is one of the reasons why the Almighty invented trigonometry. The bottom row of plates rests on the floor of the box, so the lowest disk centers sit r units above the floor, where r is the plate radius. The vertical distance between successive rows of disks is $2r \sin(60^o) = 2r\sqrt{3}/2$. We could simplify by canceling the 2's, but this obscures the origin of the formula.

```
int dense_layers(double w, double h, double r)
{
        double gap;                     /* distance between layers */

        if ((2*r) > h) return(0);

        gap = 2.0 * r * (sqrt(3)/2.0);
        return( 1 + floor((h-2.0*r)/gap) );
}
```

The number of disks which fit in the box is a function of both the number of rows of plates, and how many plates fit in each row. We always pack the bottom (or zeroth) row starting from the left-hand side of the box, so it contains as many disks as possible constrained by the width of the box. The disks in odd-numbered rows are offset by r, and we might have to remove the last disk from these rows unless there is enough slack ($\geq r$) to accommodate it:

```
int plates_per_row(int row, double w, double r)
{
        int plates_per_full_row;        /* plates in full/even row */

        plates_per_full_row = floor(w/(2*r));

        if ((row % 2) == 0) return(plates_per_full_row);

        if (((w/(2*r))-plates_per_full_row) >= 0.5)   /* odd row full */
                return(plates_per_full_row);
        else
                return(plates_per_full_row - 1);
}
```

Determining how many plates sit on a given plate can be simplified through proper use of our coordinate systems. In an unbounded lattice, two plates in row $r+1$ sit on top of a plate at hexagonal-(r, c), namely, $(r, c-1)$ and (r, c). In general, $i+1$ such plates sit in row $r+i$. However, we must clip these off to reflect the limits of our region. This clipping is easier done in array-coordinates, so we convert to determine the number of plates in our truncated cone:

```
int plates_on_top(int xh, int yh, double w, double l, double r)
{
        int number_on_top = 0;          /* total plates on top */
        int layers;                     /* number of rows in grid */
        int rowlength;                  /* number of plates in row */
        int row;                        /* counter */
        int xla,yla,xra,yra;            /* array coordinates */

        layers = dense_layers(w,l,r);
```

```
        for (row=xh+1; row<layers; row++) {
            rowlength = plates_per_row(row,w,r) - 1;

            hex_to_array(row,yh-row,&xla,&yla);
            if (yla < 0) yla = 0;                   /* left boundary */

            hex_to_array(row,yh,&xra,&yra);
            if (yra > rowlength) yra = rowlength;  /* right boundary */

            number_on_top += yra-yla+1;
        }

        return(number_on_top);
}
```

12.4 Circle Packings

There is an interesting and important connection between hexagonal grids and packing circular disks. The six neighbors of each vertex v in the grid are equidistant from v, so we can draw a circle centered in v through them, as shown in Figure 12.4(r). Each such disk touches the six disks of its neighbors, as shown in Figure 12.3.

The plate packing problem asks us to evaluate two different ways to pack a collection of equal-sized disks, one with the disk centers at the vertices of a rectilinear grid, the other with their centers at the vertices of a hexagonal grid. Which leads to a denser circle packing? It is easy to evaluate both layouts using the routines we have already developed:

```
/* How many radius r plates fit in a hexagonal-lattice packed w*h box? */

int dense_plates(double w, double l, double r)
{
        int layers;                     /* number of layers of balls */

        layers = dense_layers(w,l,r);

        return (ceil(layers/2.0) * plates_per_row(0,w,r) +
                floor(layers/2.0) * plates_per_row(1,w,r) );
}


/* How many radius r plates fit in a grid-lattice packed w*h box?  */

int grid_plates(double w, double h, double r)
```

```
{
        int layers;                     /* number of layers of balls */

        layers = floor(h/(2*r));

        return (layers * plates_per_row(0,w,r));
}
```

For large enough boxes, the hexagonal packing certainly lets us pack more plates than the square grid layout. Indeed, hexagonal packing is the asymptotically densest way to pack disks, and its three-dimensional analog is the densest possible way to pack spheres.

A 4×4 box has room for 16 unit-diameter plates under the square layout versus only 14 with the hexagonal layout, due to boundary effects. But a 10×10 box fits 105 plates in the hexagonal layout, 5 more than the square, and it never looks back from there. A 100×100 box fits 11,443 hex-plates versus 10,000 in a square layout. Thus we can gain a significant advantage with the proposed packaging technology.

12.5 Longitude and Latitude

A particularly important coordinate grid is the system of longitude and latitude which uniquely positions every location on the surface of the Earth.

The lines that run east-west, parallel to the equator, are called lines of *latitude*. The equator has a latitude of 0°, while the north and south poles have latitudes of 90° North and 90° South, respectively.

The lines that run north-south are called lines of *longitude* or *meridians*. The *prime meridian* passes through Greenwich, England, and has longitude 0°, with the entire range of longitudes spanning from 180° West to 180° East.

Every location on the surface of the Earth is described by the intersection of a latitude line and a longitude line. For example, the center of the universe (Manhattan) lies at $40^o47'$ North and $73^o58'$ West.

A common problem concerns finding the shortest flying distance between two points on the surface of the Earth. A *great circle* is a circular cross-section of a sphere which passes through the center of the sphere. The shortest distance between points x and y turns out to be the arc length between x and y on the unique great circle which passes through x and y.

We refrain from working through the spherical geometry in favor of stating the result. Denote the position of point p by its longitude-latitude coordinates, $(p_{\text{lat}}, p_{\text{long}})$, where all angles are measured in radians. Then the great-circle distance between points p and q is

$$d(p,q) \quad = \quad (\sin(p_{\text{lat}})\sin(q_{\text{lat}}) + \cos(p_{\text{lat}})\cos(q_{\text{lat}})\cos(p_{\text{long}} - q_{\text{long}})(r)$$

12.6 Problems

12.6.1 Ant on a Chessboard

PC/UVa IDs: 111201/10161, **Popularity:** B, **Success rate:** high **Level:** 1

One day, an ant named Alice came upon an $M \times M$ chessboard. She wanted to explore all the cells of the board. So she began to walk along the board by peeling off a corner of the board.

Alice started at square $(1, 1)$. First, she went up for a step, then a step to the right, and a step downward. After that, she went a step to the right, then two steps upward, and then two grids to the left. In each round, she added one new row and one new column to the corner she had explored.

For example, her first 25 steps went like this, where the numbers in each square denote on which step she visited it.

25	24	23	22	21
10	11	12	13	20
9	8	7	14	19
2	3	6	15	18
1	4	5	16	17

Her 8th step put her on square $(2, 3)$, while her 20th step put her on square $(5, 4)$. Your task is to decide where she was at a given time, assuming the chessboard is large enough to accept all movements.

Input

The input file will contain several lines, each with an integer N denoting the step number where $1 \leq N \leq 2 \times 10^9$. The file will terminate with a line that contains the number `0`.

Output

For each input situation, print a line with two numbers (x,y) denoting the column and the row number, respectively. There must be a single space between them.

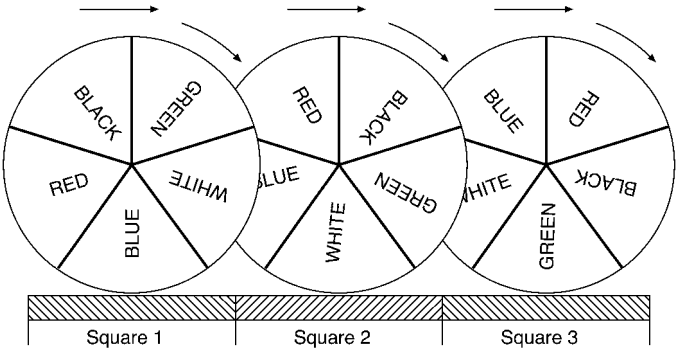
Sample Input	*Sample Output*

`8`
`20`
`25`
`0`
	`2 3`
	`5 4`
	`1 5`

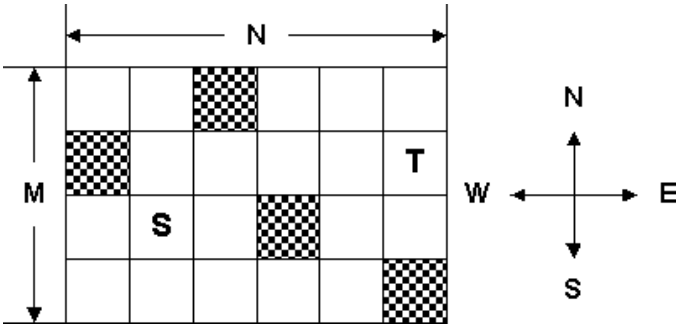
12.6.2 The Monocycle

PC/UVa IDs: 111202/10047, **Popularity:** C, **Success rate:** average **Level:** 3

A monocycle is a cycle that runs on one wheel. We will be considering a special one which has a solid wheel colored with five different colors as shown in the figure:



The colored segments make equal angles (72°) at the center. A monocyclist rides this cycle on an $M \times N$ grid of square tiles. The tiles are of a size such that moving forward from the center of one tile to that of the next one makes the wheel rotate exactly 72° around its center. The effect is shown in the above figure. When the wheel is at the center of square 1, the midpoint of its blue segment is in touch with the ground. But when the wheel moves forward to the center of the next square (square 2) the midpoint of its white segment touches the ground.



Some of the squares of the grid are blocked and hence the cyclist cannot move to them. The cyclist starts from some square and tries to move to a target square in minimum amount of time. From any square he either moves forward to the next square or he remains in the same square but turns 90° left or right. Each of these actions requires exactly 1 second to execute. He always starts his ride facing north and with the midpoint of the green segment of his wheel touching the ground. In the target square, too, the green segment must touch the ground but he does not care which direction he will be facing.

Please help the monocyclist check whether the destination is reachable and if so the minimum amount of time he will require to reach it.

Input

The input may contain multiple test cases.

The first line of each test case contains two integers M and N ($1 \leq M, N \leq 25$) giving the dimensions of the grid. Then follows the description of the grid in M lines of N characters each. The character "`#`" will indicate a blocked square, but all other squares are free. The starting location of the cyclist is marked by "`S`" and the target is marked by "`T`".

The input terminates with two zeros for M and N.

Output

For each test case first print the test case number on a separate line, as shown in the sample output. If the target location can be reached by the cyclist, print the minimum amount of time (in seconds) required to reach it in the format shown below. Otherwise print "`destination not reachable`".

Print a blank line between two successive test cases.

Sample Input

```
1 3
S#T
10 10
#S.......#
#..#.##.##
#.##.##.##
.#....##.#
##.##..#.#
#..#.##...
#......##.
..##.##...
#.###...#.
#.....###T
0 0
```

Sample Output

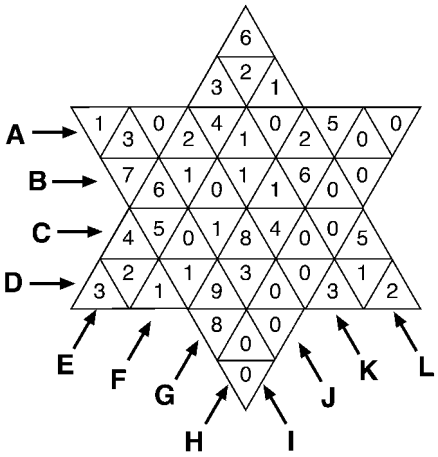
```
Case #1
destination not reachable

Case #2
minimum time = 49 sec
```

12.6.3 Star

PC/UVa IDs: 111203/10159, **Popularity:** C, **Success rate:** average **Level:** 2

A board contains 48 triangular cells. In each cell is written a digit in a range from 0 through 9. Every cell belongs to two or three lines. These lines are marked by letters from A through L. See the figure below, where the cell containing digit 9 belongs to lines D, G, and I and the cell containing digit 7 belongs to lines B and I.



For each line, we can measure the largest digit on the line. Here the largest digit for line A is 5, B is 7, E is 6, H is 0, and J is 8.

Write a program that reads the largest digit for all 12 of the depicted lines and computes the smallest and the largest possible sums of all digits on the board.

Input

Every line in the input contains 12 digits, each separated by a space. The first of these digits describes the largest digit in line A, the second in line B, and so on, until the last digit denotes the largest one in line L.

Output

For each input line, print the value of the smallest and largest sums of digits possible for the given board. These two values should appear on the same line and be separated by exactly one space. If there does not exists a solution, your program must output "`NO SOLUTION`".

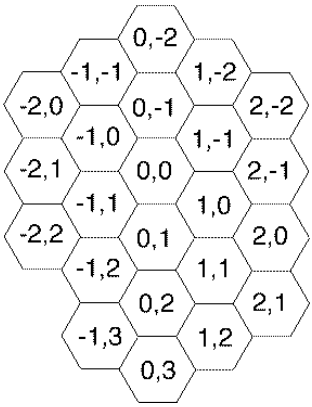
Sample Input	*Sample Output*
`5 7 8 9 6 1 9 0 9 8 4 6`	`40 172`

12.6.4 Bee Maja

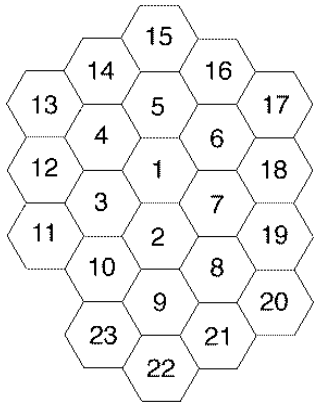
PC/UVa IDs: 111204/10182, **Popularity:** B, **Success rate:** high **Level:** 2

Maja is a bee. She lives in a hive of hexagonal honeycombs with thousands of other bees. But Maja has a problem. Her friend Willi told her where she can meet him, but Willi (a male drone) and Maja (a female worker) have different coordinate systems:

- *Maja's Coordinate System* — Maja (on left) flies directly to a special honeycomb using an advanced two-dimensional grid over the whole hive.
- *Willi's Coordinate System* — Willi (on right) is less intelligent, and just walks around cells in clockwise order starting from 1 in the middle of the hive.



Maja's system



Willi's system

Help Maja to convert Willi's system to hers. Write a program which for a given honeycomb number returns the coordinates in Maja's system.

Input

The input file contains one or more integers each standing on its own line. All honeycomb numbers are less than 100,000.

Output

Output the corresponding Maja coordinates for Willi's numbers, with each coordinate pair on a separate line.

Sample Input	*Sample Output*
1	0 0
2	0 1
3	-1 1
4	-1 0
5	0 -1

12.6.5 Robbery

PC/UVa IDs: 111205/707, **Popularity:** B, **Success rate:** average **Level:** 3

Inspector Robostop is very angry. Last night, a bank was robbed and the robber escaped. As quickly as possible, all roads leading out of the city were blocked, making it impossible for the robber to escape. The inspector then asked everybody in the city to watch out for the robber, but the only messages he got were "We don't see him."

Robostop is determined to discover exactly how the robber escaped. He asks you to write a program which analyzes all the inspector's information to find out where the robber was at any given time.

The city in which the bank was robbed has a rectangular shape. All roads leaving the city were blocked for a certain period of time t, during which several observations of the form "The robber isn't in the rectangle R_i at time t_i" were reported. Assuming that the robber can move at most one unit per time step, try to find the exact position of the robber at each time step.

Input

The input file describes several robberies. The first line of each description consists of three numbers W, H, and t ($1 \leq W, H, t \leq 100$), where W is the width, H the height of the city, and t is the length of time during which the city is locked.

The next line contains a single integer n ($0 \leq n \leq 100$), where n is the number of messages the inspector received. The next n lines each consist of five integers t_i, L_i, T_i, R_i, B_i, where t_i is the time at which the observation has been made ($1 \leq t_i \leq t$), and L_i, T_i, R_i, B_i are the left, top, right, and bottom, respectively, of the rectangular area which has been observed. The point $(1, 1)$ is the upper-left-hand corner, and (W, H) is the lower-right-hand corner of the city. The messages mean that the robber was not in the given rectangle at time t_i.

The input is terminated by a test case starting with $W = H = t = 0$. This case should not be processed.

Output

For each robbery, output the line "`Robbery #`k`:`", where k is the number of the robbery. Then, there are three possibilities:

If it is impossible that the robber is still in the city, output "`The robber has escaped.`"

In all other cases, assume that the robber is still in the city. Output one line of the form "`Time step` τ`: The robber has been at` x`,`y`.`" for each time step in which the exact location can be deduced, and x and y are the column and row, respectively, of the robber in time step τ. Output these lines ordered by time τ.

If nothing can be deduced, output the line "`Nothing known.`" and hope that the inspector does not get even angrier.

Print a blank line after each processed case.

Sample Input

```
4 4 5
4
1 1 1 4 3
1 1 1 3 4
4 1 1 3 4
4 4 2 4 4
10 10 3
1
2 1 1 10 10
0 0 0
```

Sample Output

```
Robbery #1:
Time step 1: The robber has been at 4,4.
Time step 2: The robber has been at 4,3.
Time step 3: The robber has been at 4,2.
Time step 4: The robber has been at 4,1.

Robbery #2:
The robber has escaped.
```

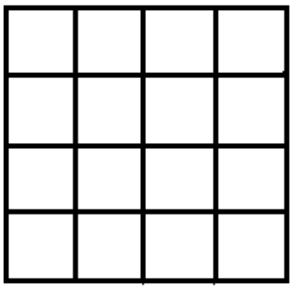
12.6.6 (2/3/4)-D Sqr/Rects/Cubes/Boxes?

PC/UVa IDs: 111206/10177, **Popularity:** B, **Success rate:** high **Level:** 2

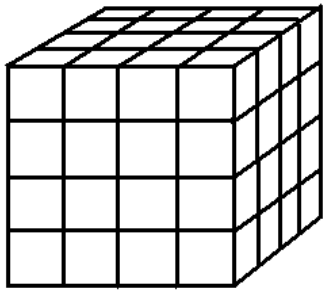
How many squares and rectangles are hidden in the 4×4 grid below? Maybe you can count it by hand for such a small grid, but what about for a 100×100 grid or even larger?

What about higher dimensions? Can you count how many cubes or boxes of different size there are in a $10 \times 10 \times 10$ cube, or how many hypercubes and hyperboxes there are in a four-dimensional $5 \times 5 \times 5 \times 5$ hypercube?

Your program needs to be efficient, so be clever. You should assume that squares are not rectangles, cubes are not boxes, and hypercubes are not hyperboxes.



A 4×4 Grid



A $4 \times 4 \times 4$ Cube

Input

The input contains one integer N ($0 \leq N \leq 100$) in each line, which is the length of one side of the grid, cube, or hypercube. In the example above $N = 4$. There may be as many as 100 lines of input.

Output

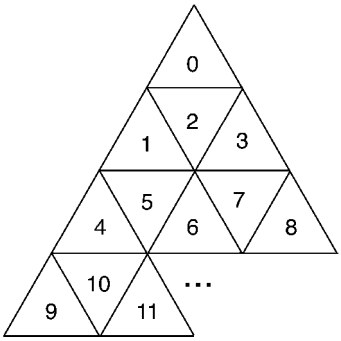
For each line of input, output six integers $S_2, R_2, S_3, R_3, S_4, R_4$ on a single line, where S_2 denotes the number of squares and R_2 the number of rectangles occurring in a two-dimensional ($N \times N$) grid. The integers S_3, R_3, S_4, R_4 denote similar quantities in higher dimensions.

Sample Input	*Sample Output*
1	1 0 1 0 1 0
2	5 4 9 18 17 64
3	14 22 36 180 98 1198

12.6.7 Dermuba Triangle

PC/UVa IDs: 111207/10233, **Popularity:** C, **Success rate:** high **Level:** 2

Dermuba Triangle is the universally-famous flat and triangular region in the L-PAX planet in the Geometria galaxy. The people of Dermuba live in equilateral-triangular fields with sides exactly equal to 1 km. Houses are always built at the circumcenters of the triangular fields. Their houses are numbered as shown in the figure below.



When Dermubian people visit each other, they follow the shortest path from their house to the destination house. This shortest path is obviously the straight-line distance that connects these two houses. Now comes your task. You have to write a program which computes the length of the shortest path between two houses given their house numbers.

Input

The input consists of several lines with two non-negative integer values n and m which specify the start and destination house numbers, where $0 \leq n, m \leq 2{,}147{,}483{,}647$.

Output

For each line in the input, print the shortest distance between the given houses in kilometers rounded off to three decimal places.

Sample Input	*Sample Output*
`0 7`	`1.528`
`2 8`	`1.528`
`9 10`	`0.577`
`10 11`	`0.577`

12.6.8 Airlines

PC/UVa IDs: 111208/10075, **Popularity:** C, **Success rate:** high **Level:** 3

A leading airline has hired you to write a program that answers the following query: given lists of city locations and direct flights, what is the minimum distance a passenger needs to fly to get from one given city to another? The city locations are specified by latitude and longitude.

To get from a city to another a passenger may take a direct flight if one exists; otherwise he must take a sequence of connecting flights.

Assume that if a passenger takes a direct flight from X to Y he never flies more than the geographical distance between X and Y. The geographical distance between two locations X and Y is the length of the geodetic line segment connecting X and Y. The geodetic line segment between two points on a sphere is the shortest connecting curve lying entirely on the surface of the sphere. Assume that the Earth is a perfect sphere of radius exactly 6,378 km, and that the value of π is approximately 3.141592653589793. Round the geographical distance between every pair of cities to the nearest integer.

Input

The input may contain multiple test cases. The first line of each test case contains three integers $N \leq 100$, $M \leq 300$, and $Q \leq 10{,}000$, where N indicates the number of cities, M represents the number of direct flights, and Q is the number of queries.

The next N lines contain the list of cities. The ith of these lines contains a string c_i followed by two real numbers lt_i and ln_i, representing the city name, latitude, and longitude, respectively. The city name will be at most 20 characters and will not contain white-space characters. The latitude will be between -90° (South Pole) and $+90^{\text{o}}$ (North Pole). The longitude will be between -180° and $+180^{\text{o}}$, where negative (positive) numbers denote locations west (east) of the meridian passing through Greenwich, England.

The next M lines contain the direct flight list. The ith of these lines contains two city names a_i and b_i, indicating that there exists a direct flight from city a_i to city b_i. Both city names will occur in the city list.

The next Q lines contain the query list. The ith of these lines will contain city names a_i and b_i asking for the minimum distance a passenger needs to fly to get from a_i to b_i. Be assured that a_i and b_i are not equal and both city names will occur in the city list.

The input will terminate with three zeros for N, M, and Q.

Output

For each test case, first output the test case number (starting from 1) as shown in the sample output. Then for each input query, print a line giving the shortest distance (in km) a passenger needs to fly to get from the first city (a_i) to the second one (b_i). If there exists no route form a_i to b_i, just print the line "`no route exists`".

Print a blank line between two consecutive test cases.

Sample Input

```
3 4 2
Dhaka 23.8500 90.4000
Chittagong 22.2500 91.8333
Calcutta 22.5333 88.3667
Dhaka Calcutta
Calcutta Dhaka
Dhaka Chittagong
Chittagong Dhaka
Chittagong Calcutta
Dhaka Chittagong
5 6 3
Baghdad 33.2333 44.3667
Dhaka 23.8500 90.4000
Frankfurt 50.0330 8.5670
Hong_Kong 21.7500 115.0000
Tokyo 35.6833 139.7333
Baghdad Dhaka
Dhaka Frankfurt
Tokyo Hong_Kong
Hong_Kong Dhaka
Baghdad Tokyo
Frankfurt Tokyo
Dhaka Hong_Kong
Frankfurt Baghdad
Baghdad Frankfurt
0 0 0
```

Sample Output

```
Case #1
485 km
231 km

Case #2
19654 km
no route exists
12023 km
```

12.7 Hints

12.1 Do we need to walk off the entire path explicitly, or can we compute the final square via some formula?

12.2 What is the right underlying graph to capture the color structure?

12.3 Can we compute the upper and lower bounds for each digit in isolation?

12.4 If we cannot find a formula to compute locations under Willi's system, how can we best simulate his traversal using an explicit data structure?

12.5 What is the right underlying graph to capture both time and space?

12.6 How do the 2-D and 3-D face incidence formulas generalize to 4-D? Is every hypercube still specified by two corner points?

12.7 How do we convert between our previous triangular coordinate systems and this new one?

12.8 Do the distances derived from your longitude/lattitude computations make sense, or is there a bug? What is the underlying graph problem?